SOFTWARE ENGINEERING

Video Call in Django with WebRTC and Channels

BLOGGER NEPAL OCTOBER 10, 2021

Hello and Welcome again, in this article, we will create a simple video calling application with Django, Django Channels, and WebRTC. Django is a high-level Python web framework. Channels is a project that takes Django and extends its abilities beyond HTTP - to handle WebSockets. WebRTC is an open standard for real-time, plugin-free video, audio, and data communication. WebRTC enables peer-to-peer communication, it allows communication among browsers and devices.



Three simple ways to size your solar system
Size by:
Application requi

Prerequisite

You should have a basic understanding of the Django framework. Having the knowledge of Django Channels is good. Some Basic understanding of JavaScript. Having the knowledge of Promises and DOM is good. If you are here to understand the flow of WebRTC and how it works, you can follow it thoroughly along as well.

WebRTC

Web Real-Time Communication is a technology that allows Web applications and sites to capture audio and/or video media. WebRTC allows Web applications and sites to exchange data between browsers without requiring any intermediary.

WebRTC establishes connections between two peers and is represented by the RTCPeerConnection interface. Once the connection is established, media streams and/or data channels can be added to the connection. Connections between peers can be made without requiring any special drivers or plug-ins, and can **often** be made without any intermediary servers.

For more see on WebRTC - Video Call with SocketIO Nodejs

Video Call with Django

There is no way we can share all of the code here, so I will try to share some important parts and share the GitHub for the project.

We first start a Django project, 'videocall', then start an app. let's call it 'call'. Then we will define a view that will serve as a static template. Then define the url to get

Three simple ways to size your solar system
Size by:
Application requi

Update your settings.py to include the 'call' and 'channels'. Then, add about ASGI. Let's use InMemoryChannelLayer as BACKEND channel layer.

```
...
...
...
INSTALLED_APPS = [
    ...
    ...
    'call',
    'channels',
]
...
...
...

ASGI_APPLICATION = 'videocall.asgi.application'

CHANNEL_LAYERS={
    "default": {
        "BACKEND": "channels.layers.InMemoryChannelLayer"
    }
}
```

Now, create the 'consumer.py'

```
# call/consumers.py
import json
from asgiref.sync import async_to_sync
from channels.generic.websocket import WebsocketConsumer

class CallConsumer(WebsocketConsumer):
    def connect(self):
        self.accept()

        # response to client, that we are connected.
        self.send(text_data=json.dumps({
            'type': 'connection',
            'data': {
                'message': "Connected"
```

Three simple ways to size your solar system
Size by:
Application requi

```
        # Leave room group
        async_to_sync(self.channel_layer.group_discard)(
            self.my_name,
            self.channel_name
        )

    # Receive message from client WebSocket
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        # print(text_data_json)

        eventType = text_data_json['type']

        if eventType == 'login':
            name = text_data_json['data']['name']

            # we will use this as room name as well
            self.my_name = name

            # Join room
            async_to_sync(self.channel_layer.group_add)(
                self.my_name,
                self.channel_name
            )

        if eventType == 'call':
            name = text_data_json['data']['name']
            print(self.my_name, "is calling", name);
            # print(text_data_json)


            # to notify the callee we sent an event to the group name
            # and their's groun name is the name
            async_to_sync(self.channel_layer.group_send)(
                name,
                {
                    'type': 'call_received',
                    'data': {
                        'caller': self.my_name,
                        'rtcMessage': text_data_json['data']['rtcMessage']
                    }
                }
            )

        if eventType == 'answer_call':
            # has received call from someone now notify the calling user
            # we can notify to the group with the caller name
```

Three simple ways to size your solar system
Size by:
Application requi

```
            async_to_sync(self.channel_layer.group_send)(
                caller,
                {
                    'type': 'call_answered',
                    'data': {
                        'rtcMessage': text_data_json['data']['rtcMessage']
                    }
                }
            )

        if eventType == 'ICEcandidate':

            user = text_data_json['data']['user']

            async_to_sync(self.channel_layer.group_send)(
                user,
                {
                    'type': 'ICEcandidate',
                    'data': {
                        'rtcMessage': text_data_json['data']['rtcMessage']
                    }
                }
            )

    def call_received(self, event):

        # print(event)
        print('Call received by ', self.my_name )
        self.send(text_data=json.dumps({
            'type': 'call_received',
            'data': event['data']
        }))

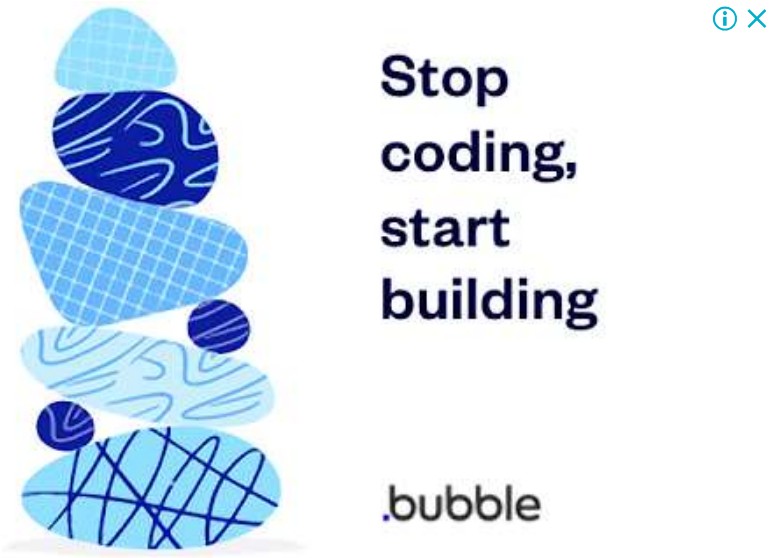

    def call_answered(self, event):

        # print(event)
        print(self.my_name, "'s call answered")
        self.send(text_data=json.dumps({
            'type': 'call_answered',
            'data': event['data']
        }))

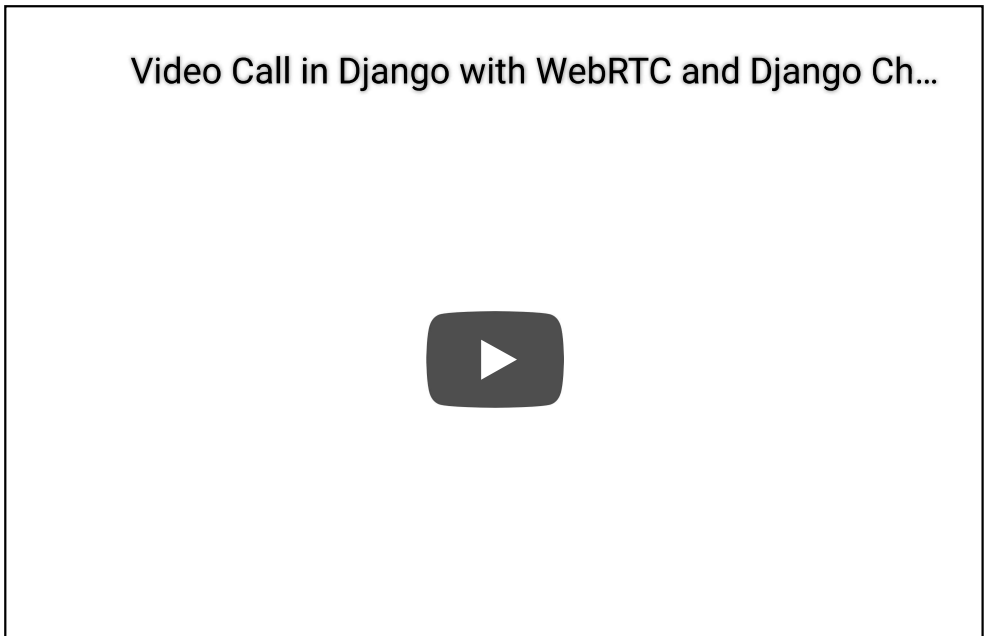

    def ICEcandidate(self, event):
        self.send(text_data=json.dumps({
            'type': 'ICEcandidate'
```

Three simple ways to size your solar system
Size by:
Application requi

In the above consumers.py, we create a CallConsumer, which will interact with the frontend. The channel is mainly responsible to transfer the "Offer", "Answer" and "ICECandidates".



The receive method will be triggered for any event from the frontend, then based on the event type we will perform the necessary actions.



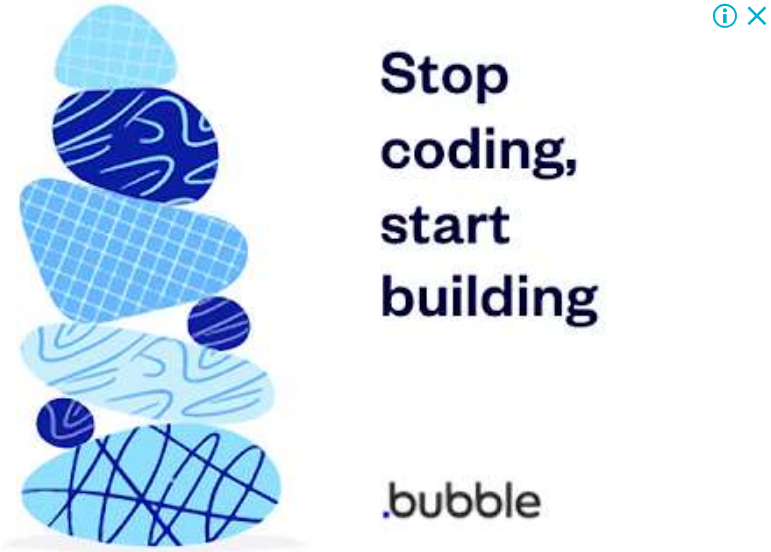


```
from django.urls import re_path

from . import consumers

websocket_urlpatterns = [
    re_path(r'ws/call/', consumers.CallConsumer.as_asgi()),
]
```

In the routing, we define the request to ws/call/ to be sent to the callConsumer.



And finally, create the 'asgi.py' on the project's app.

```
# videocall/asgi.py
import os

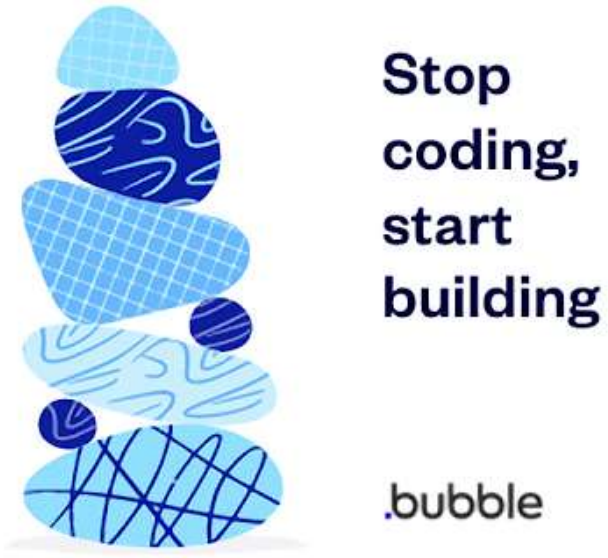
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from django.core.asgi import get_asgi_application
import call.routing

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "videocall.settings")

# application = get_asgi_application()
application = ProtocolTypeRouter({
```

Three simple ways to size your solar system
Size by:
Application requi

```
        )
    ),
})
```



In the asgi, we provide two options http and websocket. and based on the protocol of the request, it will pass the request to the respective section.

JavaScript part

```
function connectSocket() {

    callSocket = new WebSocket(
        'ws://'
        + window.location.host
        + '/ws/call/'
    );

    callSocket.onopen = event =>{
    //let's send myName to the socket
        callSocket.send(JSON.stringify({
            type: 'login',
            data: {
```

Three simple ways to size your solar system
Size by:
Application requi

```
callSocket.onmessage = (e) =>{
    let response = JSON.parse(e.data);

    // console.log(response);

    let type = response.type;

    if(type == 'connection') {
        console.log(response.data.message)
    }

    if(type == 'call_received') {
        // console.log(response);
        onNewCall(response.data)
    }

    if(type == 'call_answered') {
        onCallAnswered(response.data);
    }

    if(type == 'ICEcandidate') {
        onICECandidate(response.data);
    }
}

const onNewCall = (data) =>{
    //when other called you
    //show answer button

    otherUser = data.caller;
    remoteRTCMessage = data.rtcMessage

    // document.getElementById("profileImageA").src = baseURL + callerProfile
    document.getElementById("callerName").innerHTML = otherUser;
    document.getElementById("call").style.display = "none";
    document.getElementById("answer").style.display = "block";
}

const onCallAnswered = (data) =>{
    //when other accept our call
    remoteRTCMessage = data.rtcMessage
    peerConnection.setRemoteDescription(new RTCSessionDescription(remoteRTCMe

    document.getElementById("calling").style.display = "none";

    console.log("Call Started. They Answered");
    // console.log(pc);
```

Three simple ways to size your solar system
Size by:
Application requi

```
        const onICECandidate = (data) =>{
            // console.log(data);
            console.log("GOT ICE candidate");

            let message = data.rtcMessage

            let candidate = new RTCIceCandidate({
                sdpMLineIndex: message.label,
                candidate: message.candidate
            });

            if (peerConnection) {
                console.log("ICE candidate Added");
                peerConnection.addIceCandidate(candidate);
            } else {
                console.log("ICE candidate Pushed");
                iceCandidatesFromCaller.push(candidate);
            }

        }

    }
```

The above code section will create a connection to the WebSocket (Django channels). Then it listens to the events from the socket. based on the event type, it handles the UI and data.





We need to send some events to the backend as well. we need to send "Call", "Answer" and "ICECandidates"

```
function sendCall(data) {
    //to send a call
    console.log("Send Call");

    // socket.emit("call", data);
    callSocket.send(JSON.stringify({
        type: 'call',
        data
    }));

    document.getElementById("call").style.display = "none";
    // document.getElementById("profileImageCA").src = baseURL + otherUserProfile
    document.getElementById("otherUserNameCA").innerHTML = otherUser;
    document.getElementById("calling").style.display = "block";
}

function answerCall(data) {
    //to answer a call
    // socket.emit("answerCall", data);
    callSocket.send(JSON.stringify({
        type: 'answer_call',
        data
    }));
    callProgress();
}

function sendICEcandidate(data) {
    //send only if we have caller, else no need to
    console.log("Send ICE candidate");
    // socket.emit("ICEcandidate", data)
    callSocket.send(JSON.stringify({
        type: 'ICEcandidate',
        data
    }));

}
```



Interserver Web Hosting Review 2021. Details, Pricing & Features

blog.infointermedia.com

We define the methods, will be called as needed. The send call will be called when the user enters who to call and presses the call button. The answerCall will be called, when the user get the call and have to accept the call.

sendICECandidate(), will be called whenever the Connection get the ICE candidates, we will talk about this letter.

STUN is on the public internet, the apps can use STUN server to discover its IP:port from a public perspective. STUN server simply check the IP:Port of the incoming request and respond it back, there is not much work there is so not much powerful server is required.

If WebRTC cannot establish a connection with the above methods, TURN servers can be used as a fallback, relaying data between endpoints.

You can also go thoroughly on: Setup STUN and TURN server on Ubuntu



```
let pcConfig = {
    "iceServers":
        [
            { "url": "stun:stun.jap.bloggernepal.com:5349" },
            {
                "url": "turn:turn.jap.bloggernepal.com:5349",
                "username": "guest",
                "credential": "somepassword"
            }
        ]
};

// Set up audio and video regardless of what devices are present.
let sdpConstraints = {
    offerToReceiveAudio: true,
    offerToReceiveVideo: true
};

function beReady() {
    return navigator.mediaDevices.getUserMedia({
        audio: true,
        video: true
    })
        .then(stream => {
            localStream = stream;
            localVideo.srcObject = stream;

            return createConnectionAndAddStream()
        })
        .catch(function (e) {
            alert('getUserMedia() error: ' + e.name);
        });
}

function createConnectionAndAddStream() {
    createPeerConnection();
    peerConnection.addStream(localStream);
    return true;
}

function processCall(userName) {
    peerConnection.createOffer((sessionDescription) => {
        peerConnection.setLocalDescription(sessionDescription);
        sendCall({
            name: userName,
            rtcMessage: sessionDescription
        })
```




```
function processAccept() {

    peerConnection.setRemoteDescription(new RTCSessionDescription(remoteRTCMessag
    peerConnection.createAnswer((sessionDescription) => {
```

Women's Cycling Jersey New IN

Lightinthebox.com

```
        peerConnection.setLocalDescription(sessionDescription);

        answerCall({
            caller: otherUser,
            rtcMessage: sessionDescription
        })

    }, (error) => {
        console.log("Error");
    })
}

////////////////////////////////////////////////////

function createPeerConnection() {
    try {
        peerConnection = new RTCPeerConnection(pcConfig);
        // peerConnection = new RTCPeerConnection();
        peerConnection.onicecandidate = handleIceCandidate;
        peerConnection.onaddstream = handleRemoteStreamAdded;
        peerConnection.onremovestream = handleRemoteStreamRemoved;
        console.log('Created RTCPeerConnnection');
        return;
    } catch (e) {
        console.log('Failed to create PeerConnection, exception: ' + e.message);
        alert('Cannot create RTCPeerConnection object.');
        return;
```

Three simple ways to size your solar system

Size by:
Application requi

```
        // console.log('icecandidate event: ', event);
        if (event.candidate) {
            console.log("Local ICE candidate");
            // console.log(event.candidate.candidate);

            sendICEcandidate({
                user: otherUser,
                rtcMessage: {
                    label: event.candidate.sdpMLineIndex,
                    id: event.candidate.sdpMid,
                    candidate: event.candidate.candidate
                }
            })

        } else {
            console.log('End of candidates.');
        }
    }

    function handleRemoteStreamAdded(event) {
        console.log('Remote stream added.');
        remoteStream = event.stream;
        remoteVideo.srcObject = remoteStream;
    }

    function handleRemoteStreamRemoved(event) {
        console.log('Remote stream removed. Event: ', event);
        remoteVideo.srcObject = null;
        localVideo.srcObject = null;
    }
```

The beReady method will be called, whenever the user have to call or answer a call. The processCall, will be called to create an offer and processAccept is called whenever a user get the call(the offer), and have to create an answer.

Three simple ways to size your solar system
Size by:
Application requi

Women's Cycling Jersey New IN

Lightinthebox.com

Find the source code here: https://github.com/InfoDevkota/WebRTC-Django-Django-Channels-Video-Call

The handleIceCandidate is an listener for RTCPeerConnection, which will be fired everytime new ICE Candidates are found. Here we will simply forward it to the other user.

WebRTC not Working on Lan

Here we use the getUserMedia API to get the audio-video of the user, But some browsers restrict the use of these APIs to Secure Origins only. As Described on Deprecating Powerful Features on Insecure Origins. These APIs include:

- Geolocation
- Device motion / orientation
- EME
- getUserMedia
- AppCache

Three simple ways to size your solar system Size by: Application requi

But Lan network is not considered a secure origin. As described on Prefer Secure Origins For Powerful New Features, "Secure origins" are origins that match at least one of the following (scheme, host, port) patterns:

- (https, *, *)
- (wss, *, *)
- (*, localhost, *)
- (*, 127/8, *)
- (*, ::1/128, *)
- (file, *, —)

So you cannot test on LAN network, you should test on local, or deploy using an SSL. Follow Django Channels, WebSocket Authentication Deployment.

Conclusion

We implement a Video calling application on Django, using the Django Channels and WebRTC. There is not much to do on the backend side, as Django is used just as a signaling server, and data are transferred directly between clients.

Posted By: Sagar Devkota

IT Exams

Exam-Labs

Three simple ways to size your solar system

Size by:

Application requi

NEXT

PREVIOUS

2 Comments

Hassan December 13, 2021 at 1:56 PM

how to disconnect the video if user end the call

Reply

Anonymous April 27, 2022 at 11:03 PM

simply stop your webrtc peer connection instance

Reply

Enter Comment

Search this blog

TOOLS

Nepali Calendar Patro (BS)
Is This a Word?
Pokhara University Results
Nepali Typing
Preeti to Unicode

Three simple ways to size your solar system
Size by:
Application requi

Quiz
Note Search
SocketIO Client Online
National Languages
Learn Gurung
This Year so Far
IPO Result of Multiple Users

2022 (9)

2021 (95)

2020 (36)

2019 (81)

2018 (55)

FEATURED POST



Tricks and Tips

Dependent Mindset- Seeking for Help in Each Step at Work

Three simple ways to size your solar system
Size by:
Application requi

About us

Blogger Nepal: A tech blog for developers out there, Here we focus on writing contents to help beginners to take a step ahead and learn about advanced concepts. We will write on Software Engineering, Nodejs and WebRTC. We occasionally write about Environment and Forestry as well.

Labels

Academic Tricks and Tips Opinion Do Online Tour & Travel Software Engineering Earning Forestry Facebook Internet Geomatics Productivity Quiz Business Games NTC Social media Affiliate Marketing Agriculture Google Adsense IOF Notes Marketing Top/Best Namaste Pay

Pages

- Home
- About Us
- Contact Us
- Terms And Conditions
- Privacy Policy

© Copyright 2022 Blogger Nepal. Designed by Blogger templates.

Three simple ways to size your solar system
Size by:
Application requi